Technical Assessment: AI-Driven Fullstack Developer Intern

1. Objective

Build a small Warehouse Receiving System using AI-assisted, specification-driven development.

You are expected to act as a fullstack developer who can translate business requirements into technical specifications, use an AI coding agent to implement those specifications, verify the generated application, and deploy it.

The purpose of this assessment is not to measure how much code you can write manually. We want to understand how you define requirements, structure technical documentation, direct AI agents, evaluate their output, and deliver reliable software.

2. Business Scenario

A warehouse currently records incoming inventory using spreadsheets. The warehouse team wants a web application to register incoming deliveries, record received quantities, and track available stock.

The system must support:

- Managing products and their SKU codes.
- Creating inbound receiving documents.
- Adding products and expected quantities to receiving documents.
- Recording actual received quantities.
- Tracking receiving document statuses.
- Viewing available inventory.
- Viewing inventory transaction history.

The application must prevent users from receiving more than the remaining expected quantity. A receiving document must not be completed unless all its items have been fully received.

Assume that the application is used by warehouse operators and warehouse supervisors.

*The business requirements are unclear, document your assumptions and explain your decisions.

3. Required Deliverables

A. Product Requirements Document (PRD)

Create a PRD that includes:

- Problem statement and business objectives.
- User roles and their responsibilities.
- Functional requirements.
- Non-functional requirements.
- User stories and acceptance criteria.
- Business rules and validation requirements.
- Application scope and out-of-scope features.
- Assumptions, constraints, and unresolved questions.

Requirements must be sufficiently specific for an AI coding agent to implement without repeatedly guessing business behavior.

B. Technical Specification

Create a technical specification derived from the PRD.

It must include:

1. **System architecture:** Components, responsibilities, and their interactions.
2. **ERD:** Entities, relationships, primary keys, foreign keys, and important constraints.
3. **Database design:** Table definitions, field types, indexes, and migration strategy.
4. **API contract:** Endpoints, HTTP methods, request/response schemas, validation, and error responses.
5. **Data flow:** How information moves through the system.
6. **API flow:** Step-by-step processing of the main receiving workflow.
7. **State transitions:** Valid receiving document status changes.
8. **Authorization:** Which roles can perform which actions.
9. **Testing strategy:** Unit, integration, and end-to-end testing requirements.

You may use Mermaid, OpenAPI, SQL DDL, or other appropriate formats.

C. Application Implementation

Build a working application using your technical specification as the primary implementation input.

Required features:

- Product listing and creation.
- Receiving document listing and creation.
- Receiving document detail with its items.
- Recording partial and full receipts.

- Automatic inventory updates.
- Inventory listing and transaction history.
- Validation of invalid operations.
- Basic authentication or an equivalent documented role-based access mechanism.

The frontend must provide a usable interface for the main workflows. The backend must enforce business rules independently of frontend validation.

D. AI Agent Workflow

Use an AI coding agent of your choice.

Your development process must be specification-driven: the agent should receive your PRD and technical documents as project inputs, rather than relying entirely on a large one-off natural-language prompt.

Submit:

- The PRD and technical specification files.
- Relevant project instruction files used by your agent.
- At least three meaningful prompts or agent task instructions.

You may use prompts during development. We want to see how you use them alongside your documentation, not prohibit them.

E. Testing and Quality Assurance

Provide:

- Automated tests for critical business rules.
- API validation and error-handling tests.
- Evidence that inventory quantities remain correct after receiving.
- Evidence that duplicate or invalid receiving operations do not corrupt stock.
- A short list of known limitations and unresolved issues.

F. Deployment

Containerize the application and provide a reproducible deployment setup.

Required:

- Dockerfile(s) and Docker Compose configuration.
- Application and database configuration through environment variables.
- Database migrations that can be executed reliably.
- Health check or readiness endpoint.
- Instructions to start the application from a clean environment.
- A demonstration that the application runs successfully.

A public deployment is optional unless specifically requested by the hiring team. Never commit credentials or secrets to the repository.

4. Technical Constraints

- Use Git and maintain a meaningful commit history.
- Use a relational database such as PostgreSQL.
- Use a documented REST API.
- Use an AI coding agent as a core part of implementation.
- You may select your preferred frontend and backend frameworks, provided the complete stack is documented and reproducible.
- You may use libraries and AI-generated code, but you are responsible for reviewing and validating the result.
- Do not implement unnecessary features beyond the required scope.

5. Submission Format

Submit a Git repository containing:

- `/docs/prd.md`
- `/docs/technical-spec.md`
- `/docs/erd.mmd` or an equivalent ERD artifact
- `/docs/openapi.yaml` or an equivalent API contract
- `/docs/data-flow.mmd` or equivalent diagrams
- `/docs/ai-workflow.md`
- `/docs/testing.md`
- Application source code
- Database migrations
- Docker configuration
- `README.md` with setup, execution, testing, and deployment instructions

You may adapt this structure if your documentation remains easy to navigate.

6. Time Limit

Target effort: 8–12 hours, with a maximum submission window of 3 calendar days.

Prioritize correctness and completeness of the core workflow over visual polish or feature quantity.

Document any unfinished work and explain what you would improve with additional time.

7. Technical Review

After submission, you will participate in a technical review session.

You may be asked to:

- Explain a design decision in your ERD or API contract.
- Trace a receiving operation from the frontend to the database.
- Modify a requirement and update the implementation.

You should be able to explain and defend the submitted work, including code generated by AI.